

For personal use only,
please do not
distribute on
non-UBC domains.

02b - Cost of Change: Cohesion

CPSC310

Nick Bradley & Kyle Chin



Learning Goals

By the end of the lecture you should be able to:

- Analyze code cohesion in terms of binding (data, logic, and order)
- Understand how cohesion affects risk of modifying code
- Describe the process of refactoring and how it can reduce risk as code evolves

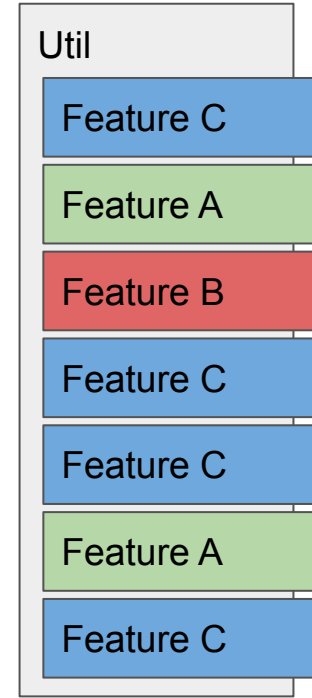
Difficulties & Risks of Code Evolution

- High coupling impacts:
 - **Difficulty:** making edits in more places
 - **Risk:** missing code that needs to be changed
- Low cohesion impacts:
 - **Difficulty:** understanding what the code is actually doing
 - **Risk:** edits can affect code unrelated to the reason for change

Cohesion Intuition

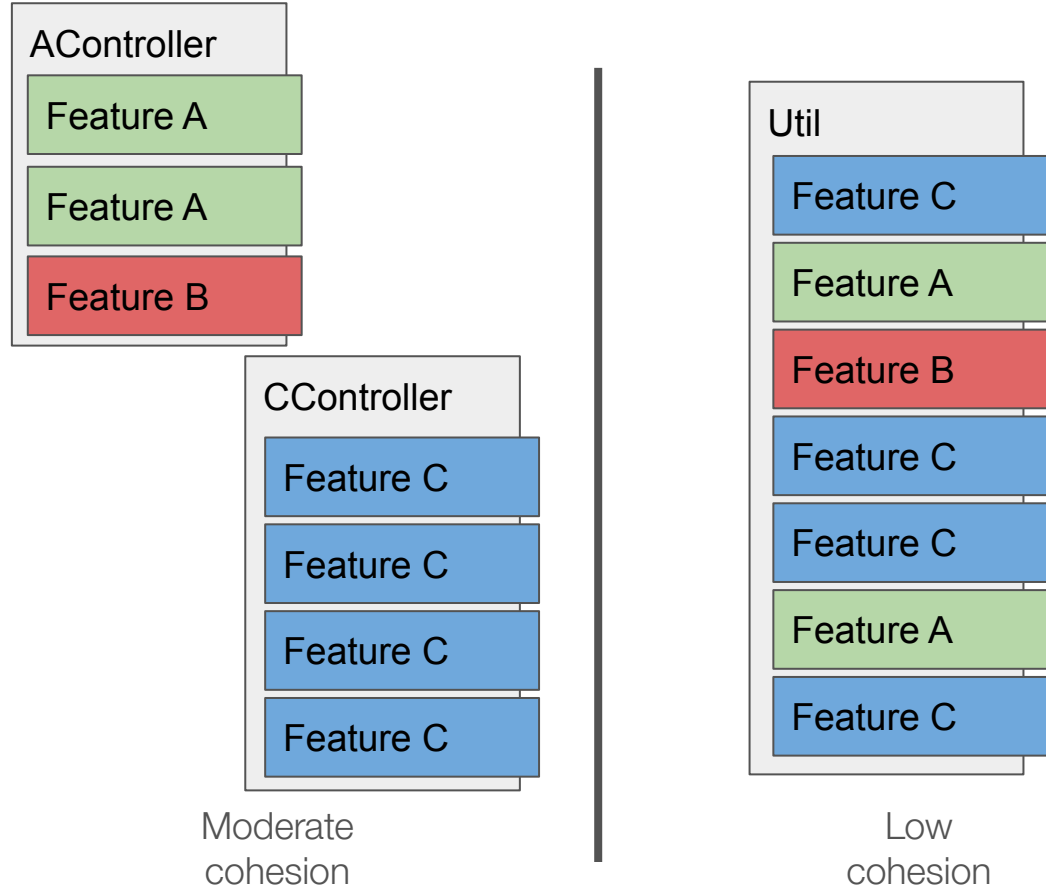
- **Cohesion** is the relationship within a unit of code (e.g. a method):
 - Units of code should contain functionality that *belong together*.
 - Harder to do as code evolves
- Measured between pairs within the unit:
 - Pairs can share *data, logic, and/or ordering*
 - The more they share, the more they belong together!

Cohesion

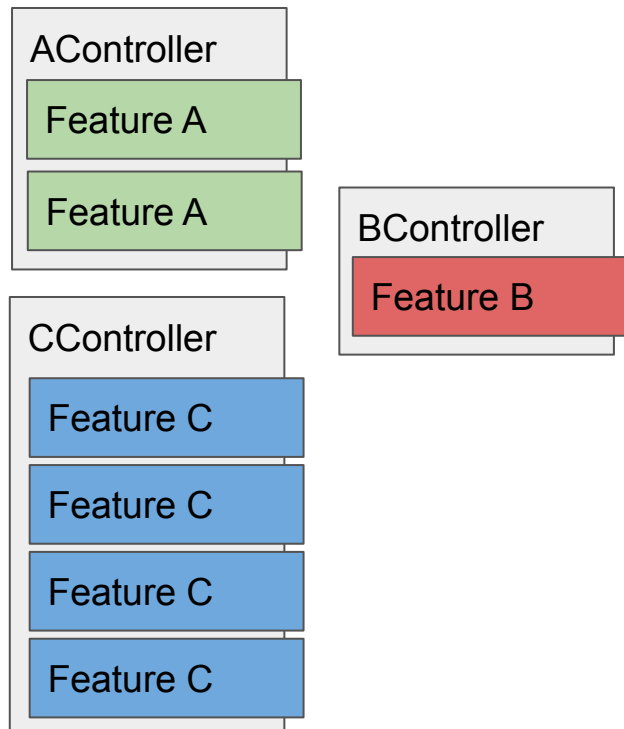


Low
cohesion

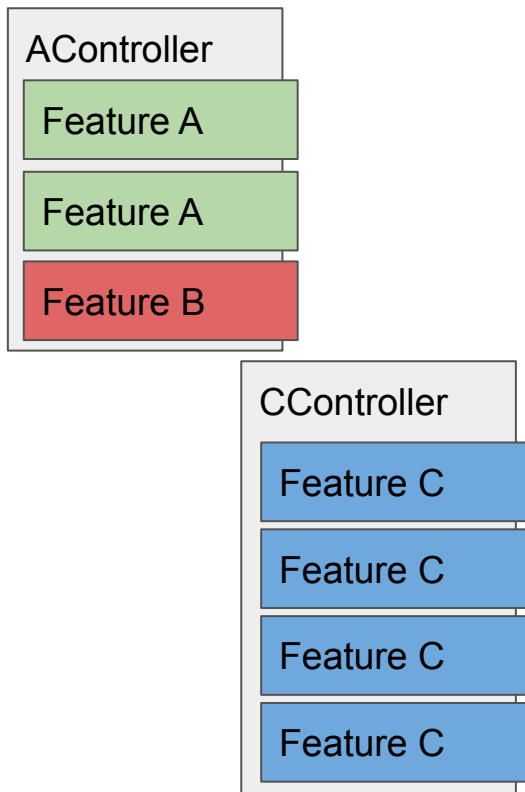
Cohesion



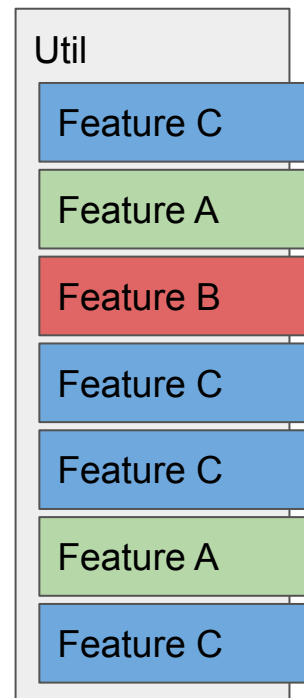
Cohesion



High
cohesion



Moderate
cohesion



Low
cohesion

Kinds of Cohesion Bindings

Data Binding: statements in the unit reads or writes the same data.

```
function summarizeReadings(readings: number[]) {  
  const total = readings.reduce((sum, r) => sum + r, 0);  
  const min = Math.min(...readings);  
  const max = Math.max(...readings);  
  return { total, min, max };  
}
```

Kinds of Cohesion Bindings

Logic Binding: statements in the unit are a branch of the same conditional (or switch/type code) and are executed *because* of that condition.

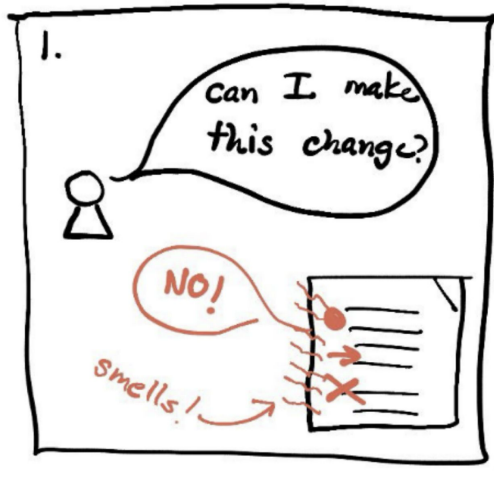
```
function process(kind: "log" | "email" | "cleanup"): void {  
  if (kind === "log") {  
    console.log("Process completed");  
  } else if (kind === "email") {  
    sendEmail("Alert triggered");           // (1)  
  } else if (kind === "cleanup") {  
    deleteFile("/tmp/cache");              // (2)  
    sendEmail("Cache cleared");             // (3)  
  }  
}
```

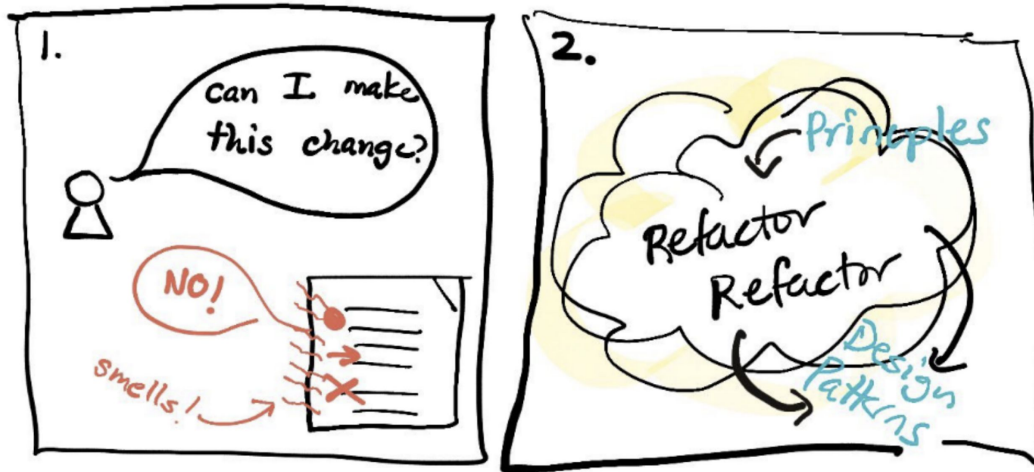
Kinds of Cohesion Bindings

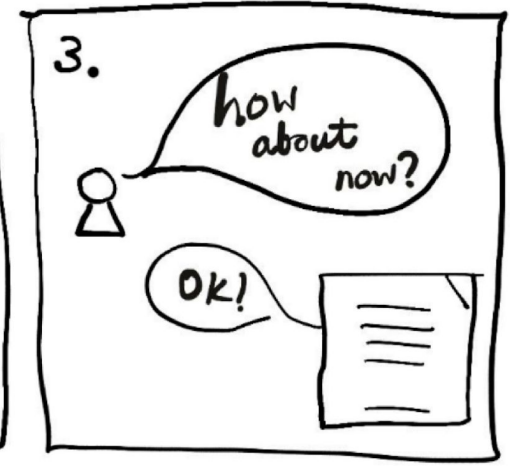
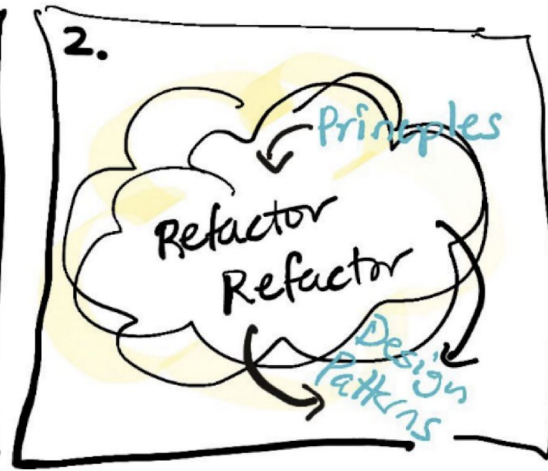
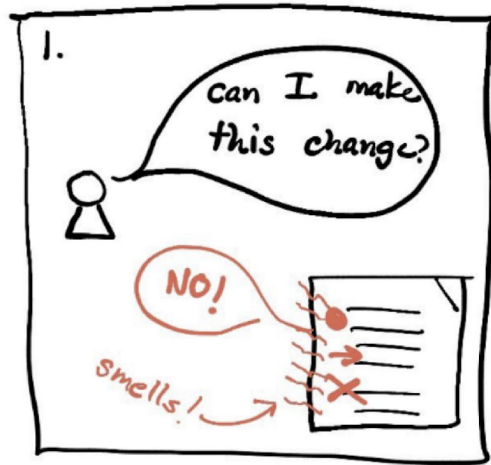
Order Bindings: statements in the unit must run in a fixed sequence.

```
function bootSystem(): void {  
    initializeLogger();  
    loadConfiguration();  
    connectToDatabase(); // database needs config  
    startServer();       // server needs db  
}
```

Refactoring







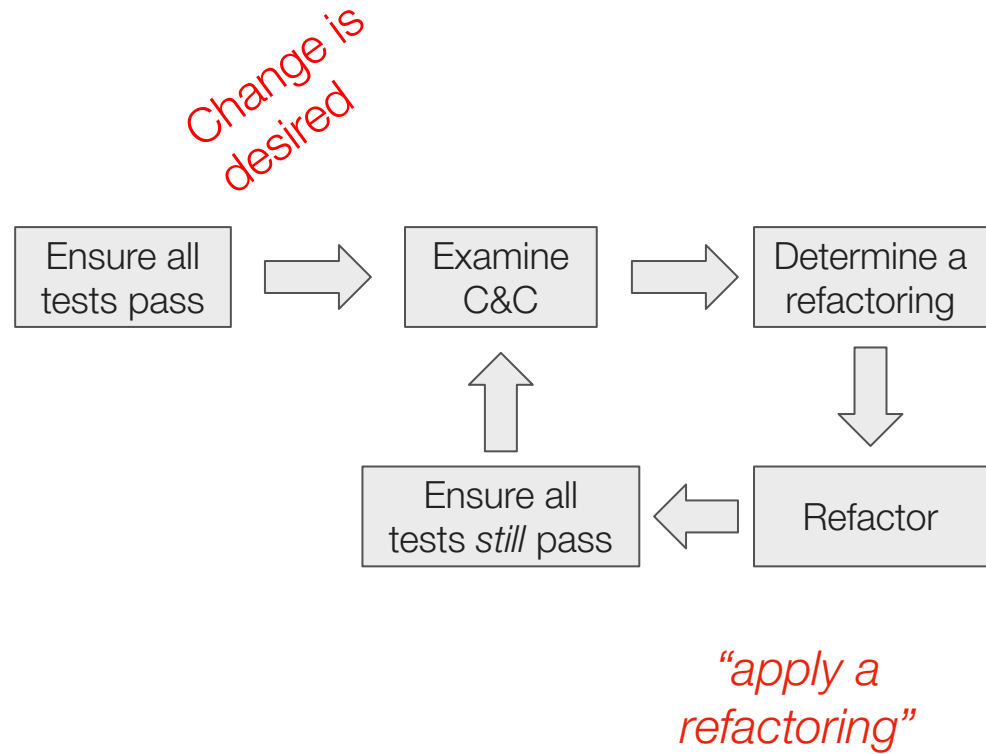
What is refactoring?

Predictable and
meaning* preserving
code transformations.

*meaning == semantics == behaviour

How to Refactor?

1. Make sure all your tests pass.
2. Examine coupling & cohesion.
3. Determine how to refactor code.
4. Apply the refactoring.
5. Run tests to make sure you didn't break anything.
6. Repeat until change is localized.



Refactoring timeline

- NOT:
 - 2 weeks of every 6 months.
 - When the tests are failing.
 - When you should just rewrite the code.
 - When you fix a bug.
- Instead, opportunistically during development:
 - If you recognize a warning sign.
 - Before/after adding a feature.
 - When you review code/your code is reviewed.

Recap: Cohesion & Refactoring

- Cohesion
 - Identifies units of code that belong together
 - Code in the units measured by what they share:
 - Data:
 - Logic:
 - Order:
 - Used together with coupling to assess difficulty and risk to evolving code
- Refactoring
 - Applied when making a change would be difficult/risky
 - Tightly coupled code and/or code with low cohesion
 - Systematic process: structure changes, behaviour preserved